

SWIFT: Semantic Web Intent Framework for Intent Translation

Paul Alcock, Revika Anand, Charalampos Rotsos, Nicholas Race
Lancaster University, UK
Email: p.alcock1@lancaster.ac.uk

Abstract—Intent-based networking (IBN) has emerged as a promising paradigm to simplify the management of network infrastructures. At the heart of IBN systems lies the intent handler, a management entity that transforms declarative network goals into imperative network configuration that meets specific policy and optimization requirements. Practical research has produced a number of intent handler implementations, that exhibit specialized use-cases with limited extensibility. This paper presents SWIFT, a framework to simplify the development of custom intent scenarios. To support this capability, SWIFT delivers two main contributions. Firstly, it employs an ontology to abstract several standardized and open network data models (3GPP Core Configuration, ETSI NFV-MANO, ONOS NBI) and support the translation of slice delivery intents, modelled using the TMForum common intent model, into effective infrastructure configurations. Secondly, SWIFT offers a reasoning framework for network operators to implement custom intent handling logic using logical programming. The reasoner can use the rich information of the ontology both to automate the translation of intent requirements, and validate intent goals based on the current network state. Our proof-of-concept SWIFT implementation can automate and optimize the delivery of private 5G network slices. Furthermore, we demonstrate the optimal planning and delivery of mobile slice intents in a real mobile network topology with minimal processing overheads.

I. INTRODUCTION

In the rapidly evolving landscape of telecommunications, network automation has emerged as a critical necessity. As the demand for high-speed, reliable, and ubiquitous connectivity continues to surge, traditional manual network management approaches, like network over-provisioning, are proving inadequate and expensive. A good example of these challenges are mobile networks. Mobile operators have invested significantly in recent years in spectrum acquisitions, and infrastructure upgrades to enable large portions of the population to access 5G services [1], [2]. In order though to realize the 5G vision, it is essential to realize fast management mechanisms that can roll out service in a matter of minutes and which can dynamically discover and use spare infrastructure capacity.

IBN has emerged as a popular approach to simplify network management and improve network automation. IBN allows users to express operational requirements through an “intent” abstraction, which captures the desired network behaviors without specifying how they should be implemented [3]. Declarative intent goals are transformed into imperative network configurations through an *intent handler*, a software component that uses contextual network information (*e.g.*, resource inventory), to generate an optimal configuration based

on the operator policies (*e.g.*, link load). Furthermore, intent-based management can support a range of automation management levels, from simple service provision to runtime service adaption using a closed control-loop mechanism, which continuously monitors and verifies intent fulfillment.

The IBN paradigm is rooted in early attempts towards autonomic network management, including the IETF ANIMA [4]. However, the network community recognized its potential in recent years with the extended standardization of network management interfaces. A big challenge for implementing IBN technologies is the significant gap between the scope and goal of intent standards and existing implementations. IBN standards, such as the TMForum intent common model [5], envision self-driven network management realized through intelligent and reusable intent handler hierarchies. In contrast, relevant research efforts, discussed in Section II, primarily deliver single-purpose intent handlers with fixed support for autonomic operation. Considering the ongoing adoption of new network technologies, like SDN, developing open and extensible tools is essential to allow the community to develop new capabilities that exploit technological potential.

This paper proposes SWIFT, a novel extensible intent handler framework, that simplifies the development of intent-handling logic and allows rapid prototyping of intent scenarios. Our system aims to provide a blueprint for the development of generic intent handlers that support easy extensibility and customization. We believe these two capabilities are essential in simplifying the transition of current network management mechanisms into higher autonomy levels by simplifying service digitalization. Toward this goal, we identify two main challenges. Firstly, the increasing adoption of network programmability creates a large offering of management data models. 5G technologies offer a good example of this challenge, where end-to-end service configuration requires coordination between the Radio Access Network (RAN) orchestrator, transport network controllers, cloud, and 5G core management services. These interfaces manage diverse resources, and although certain domain models are standardized, several still require ad-hoc integration. Current intent platforms depend on application logic to integrate models, which requires extensive code rewrites during the onboarding of new technologies. Using application code for technology integration can incur significant costs for OSS/BSS code maintenance and hinder operators’ ability to achieve agility and service digitalization. SWIFT uses Semantic Web technologies, and the Web On-

tology Language (OWL) to design an ontology that captures the relationships between domain entities, allowing intuitive association between concepts in heterogeneous data models. The ontology structures a knowledge graph, which captures domain relations without relying on external application logic.

Secondly, rapid scenario prototyping requires new development tools for intent-handling logic that simplify and abstract the programming complexity. Existing approaches use imperative scripting languages, such as Python, to implement fixed intent handler logic and programmatically converge model integration. SWIFT implements a semantic reasoner in Prolog, which allows operators to encode their intent logic as a set of logic rules and axioms over the knowledge graph. Operators can use logical rules to translate declarative intent goals into a new network configuration in the knowledge graph and implement validation logic, which detects early situations where an intent goal is not met. Additionally, the expressiveness of declarative languages can abstract model changes, enabling smoother evolution as an underlying infrastructure enables new technologies. This paper offers the following contributions:

- We present SWIFT, the first implementation of an extensible intent handler framework. It uses a semantic web ontology to combine several management models, while network operators can express their intent processing logic using Prolog predicates.
- We present a slice automation use-case that uses the SWIFT ontology and reasoner to optimally deploy edge slices on a large network infrastructure.
- We demonstrate the ability of the SWIFT framework to deploy a slice on a virtual network infrastructure by combining the data models of the ONOS controller, the OSM orchestrator, and the Free5gc 5G Core service configuration model. Our example intent scenario can plan the delivery of edge slice intents in under 80 seconds in a heavily utilized network infrastructure.

In the remainder of this paper, we elaborate on related efforts on data modeling and intent-based management in Section II. Furthermore, we present the ontology and the design of the SWIFT intent handler framework in Section IV and evaluate our proof-of-concept implementation using a 5G network slice automation scenario in Section V. Finally, we conclude our work in Section VI.

II. INTENT HANDLER AND AUTOMATION

Critical to the delivery of IBN technologies is the intent handler, a software component responsible for interpreting, processing, and assuring intents. An intent handler translates abstract service goals into configuration for the management services of the underlying network infrastructure. The intent handler, depicted in Figure 1, consists of three main components: Context, Decision, and Action. The Context component stores inventory and monitoring information of the underlying infrastructure alongside active intent objects and service blueprints. The Decision component implements algorithms that operate on the Context data to interpret and verify intent

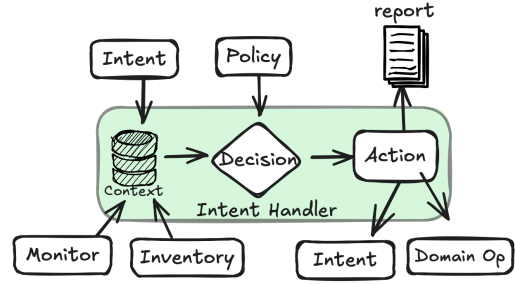


Fig. 1. Intent Handler Abstract Architecture

goals and translate intent goals into network configurations based on the current infrastructure state and operator-defined policies. Finally, the Action component deploys configuration updates using management interfaces or more granular intent interfaces in multi-layer IBN architectures. In parallel, the intent handler can support reporting goals about the intent status by transforming appropriate monitoring information. The intent handler architecture can support a range of autonomous capabilities, from fixed automation actions to self-X autonomous operation. To allow extensibility in the design of an intent handler, the platform should allow network operators to express their custom logic in each component. This can include the ability to capture different vendor and technology information in the Context component or develop different optimization strategies in the Decision component. SWIFT builds on this principle to develop a holistic environment for intent scenario development.

III. RELATED WORK

The standardization of the intent common model by the TMForum has fueled the development of intent scenarios and intent handlers. Baktir *et al.* [6] presented an IBN demo, compatible with the TMForum common model, supporting conflict resolution for mobile networks. A proof-of-concept implementation of an intent handler supporting TMForum intents was presented by the H2020 TeraFlow and SOLUTIONS projects [7], however information on the use of Semantic Web technologies is limited. Mehmood *et al.* [8] presented an intent handler for mobile networks, that extends the TMForum intent model, by adding new ontology terms for service, resource, and KPI domains. The resulting system supported only static intent planning, implemented in Python and using hand-crafted SPARQL queries.

Early attempts to bridge the semantic gap between management models (*e.g.* SNMP SMI, TMF SID) used Semantic web technologies to link information across domains [9]. Such approaches can support the functionality of the Decision component in an intent handler. The Network Markup Language (NML) [10] was the first attempt for a network control and configuration ontology, targeting Ethernet and IT technologies and offering support for additional technology extensions.

Pantuza *et al.* [11] used aspects of the NML model to program an SDN controller and automate its configuration. Similarly, the mobile ontology, developed by the SPICE ICT project [12], modeled resources and services in a mobile infrastructure. Unfortunately, the adoption of converged information models during the early days of network programmability was limited, as the proprietary nature of equipment limited the generality of such mechanisms. In parallel, new network modeling languages, like YANG [13], simplify data model and service specifications for several programming languages. Network controllers, such as ONOS [14] and ODL [15], use YANG to realize model-based component layers, simplifying code development. However, YANG remains a pure data modeling mechanism, lacking the reasoning and verification capabilities afforded to RDF graphs.

In parallel, the research community has explored the design of reasoning mechanisms for network management, that can support the Decision stage of an intent handler. Initial efforts developed declarative programming languages for network policies [16], [17]. More recently, Houidi *et al.* [18] developed a knowledge-based system for the Python Pyke reasoning engine; presenting a mechanism for automated VPN provisioning and other common network tasks. Similarly, Rotsos *et al.* [19] used an RDF scheme to model the OpenFlow protocol and implement a path computation mechanism using the built-in reasoning engine of the StarDog database. McNamara *et al.* [20] developed a static intent handler that translates intents expressed in natural language into an intent model using an AI/ML NLP mechanism. DIPS [21] closely aligns with our approach, using logical programming for intent translation. The resulting system does not support the TMForum common model and lacks the strong modeling capabilities of an ontology. [22] also defines an approach for creating orchestrator network configurations using Haskell and Category theory. Our approach is similar but differs in its close alignment with formal semantic web underpinnings.

The majority of intent handler systems adopt a fixed design approach. Their functionality prioritizes simplicity in supporting specific intent handler scenarios over extensibility, which leads to extensive boilerplate and code rewrites to introduce new capabilities. This limits the flexibility to easily enable intent-driven scenarios in new network domains or adapt to new network technologies. Early attempts, such as the NILE intent language [23], would require extended re-design to support automation scenarios in new domains, such as 5G. This is a key requirement also driven by existing intent modeling standards, such as the TMForum Intent Common Model [24] and the 3GPP Intent-driven Management Services [25], where the intent models are designed to be domain-agnostic and to support a wide range of automation capabilities and services.

IV. SWIFT ARCHITECTURE

SWIFT is an extensible intent handler framework, designed to allow easy development of new intent scenarios using logical rules. Its design aims to realize the different functional component of the intent handler architecture depicted

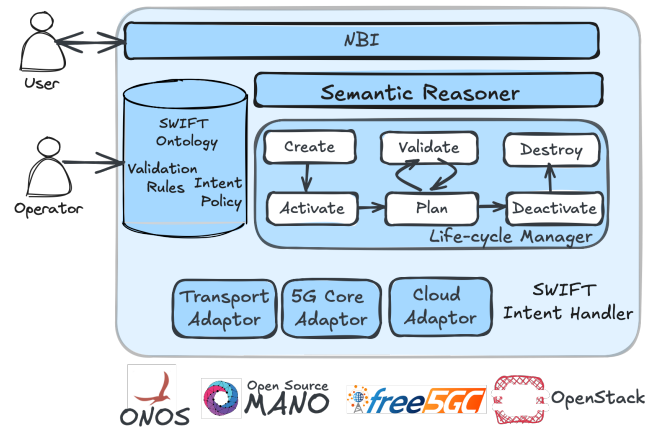


Fig. 2. SWIFT Intent Handler Architecture.

in Figure 1 In contrast to prior approaches, SWIFT provides a generic framework for operators to develop their intent-handling logic for an array of intent scenarios. To model the intent context, we develop a network ontology. This ontology is used to represent, process, and reason over data collected from management interfaces exposed by the underlying network infrastructure. Ontologies contextualize data with semantic meaning, encoding relationships between different data models as a directional knowledge graph. This makes the data more interpretable. SWIFT applies this approach to both intent goals and network infrastructure state, bridging the gap between granular network configurations and abstract intent objectives. Users can extend ontology models by injecting custom Universal Resource Identifiers (URIs) to integrate new network technologies or management models, thus allowing seamless extensibility, as discussed in Section IV-A. To implement the Decision component, we developed a Semantic Reasoner written in Prolog. The reasoner, discussed in Section IV-B, uses the ontology information as semantic facts and supports the expression of intent processing logic in the form of logical rules to transform intent goals into configuration changes across domains. Furthermore, we utilize technology adapters for the Intent Handler to interact with the network infrastructure in an effort to simplify the synchronization of the network ontology with the infrastructure state. Technology adapters are used to monitor both resource availability and inferred configuration updates to the network infrastructure, as discussed in Section IV-C. We conclude this section by discussing the implementation of SWIFT and presenting an intent scenario for automated slice deployment.

A. An ontological approach to IBN

Modern network infrastructures use multiple information domains to support programmability in the management layer, each offering diverse services and resources. Orchestrating configurations across domains to meet management goals requires mechanisms that compose and abstract various data models. This integration is typically achieved through appli-

cation code, as much of the semantic meaning of a model is not represented within the model itself. As relations between data fields are not explicit, contextual relationships between network concepts must be asserted in application logic, adding boilerplate code and burdening code maintenance as models are modified and introduced. SWIFT uses a semantic ontology to provide a shared vocabulary to describe and contextualize various network concepts, revealing the semantic meaning of information inside a data model. This enables decision processes to associate data points and infer information that enables integration across domains for end-to-end service delivery. For example, to deploy a slice in a 5G infrastructure, the intent handler needs to stitch together configuration in the radio, transport, cloud, and 5G core domains. SWIFT adapters synchronize the network state with the ontology, producing a knowledge graph that can be manipulated programmatically.

The semantic ontology of our slice deployment intent is depicted in Figure 3. The ontology models the configuration parameters in the transport, cloud, and 5G Core domains to support slice deployment. The ontology is organized using 4 distinct namespaces: intent, cloud, network, and service. The intent namespace models intent with a collection of `DeliveryExpectation` objects, similar to the TMForum Intent Common Model [5]. For our intent scenario, we model two aspects of a slice deployment: a slice uptime expectation, which controls the replication of 5G core function instances that underpin slice delivery (i.e. UPF and SMF), and a slice bandwidth expectation, which influences the path selection in the transport and User Plane Function (UPF) configuration. The network namespace models the transport topology, borrowing modelling concepts from the ONOS topology model. Specifically, a transport topology is modeled as a collection of `NetworkSwitches` offering `Interfaces (INTF)` objects, connected by `Wire` objects with specific `Bandwidth` characteristics. To associate bandwidth delivery expectations with the backhaul topology, we use a `NetworkSlice` object to store a list of bandwidth `Claims` on point-to-point `Wires` and form an end-to-end `Path` object.

The cloud namespace models the cloud infrastructure of the operators as a list of `PhysicalMachines` with specific resources (`CPU`, `Memory`, `Storage`) and a connection to the network through multiple `NIC` objects. Each `PhysicalMachine` manages a set of `Hypervisor` or `ContainerEngine` instances with specific virtual resources (`vCPU`, `vMemory`, `vStorage`). Function instances are modeled as `DeploymentUnits`, while connectivity between functions is modeled using the `VirtualSwitch` object. Finally, the service namespace models the 5G service using a modeling approach similar to the NFV-MANO model, where a network service is composed of a list of `Virtual Network Functions`. Specifically, a service is modeled as a collection of `FunctionSpec` objects with specific `DeploymentSpec`, which relate to a `DeploymentUnit`.

Our ontology structure is encoded using the Web Ontology Language (OWL) [26], a W3C standard that is used to represent the structure of complex information. An OWL

```
capableHost(Host, FuncSpec) :-
    host(Host),
    hasVCPUCapacity(Host, FuncSpec),
    hasMemoryCapacity(Host, FuncSpec),
    hasStorageCapacity(Host, FuncSpec).

deploymentUnit(DepU) :-
    virtualMachine(DepU);
    container(DepU);
    namespace(DepU).
```

Listing 1: Example SWIFT predicates that query the knowledge graph data.

ontology can validate the well-formedness of data structure, as well as relation inference between concepts. The knowledge graph data are serialized using RDF [27], a W3C standard for representing complex information using triples, composed of three parts: subject (the resource being described), predicate (the subject's relation to the object), and the object (another resource).

It is worth highlighting that semantic ontologies can easily extend the ontology with new concepts, without having to rewrite code. A user can include new concepts in an ontology by referencing the Universal Resource Identifiers (URIs) of a different ontology. Furthermore, the presented SWIFT ontology can be extended by users or even replaced by a different set of namespaces, to support the specific network automation requirements and technologies.

B. Semantic Reasoning for Intent Validation and Translation

SWIFT offers a semantic reasoner to implement intent processing logic in the form of Prolog predicates. Prolog is a logic programming language that is particularly well-suited for tasks that involve symbolic reasoning, pattern matching, and rule-based logic. Predicates define rules that model objects and relationships in a format similar to RDF triplestores and graph databases. As relationships between concepts are baked into the model itself, users can express complex network state queries using only a few lines of Prolog code. Two example predicates that operate on the knowledge graph are depicted in Listing 1. The `capableHost` predicate queries the knowledge graph for any execution environment which is capable of hosting a network function based on the specification `FuncSpec` (i.e. container runtime, baremetal, hypervisor). This predicate demonstrates the expressiveness of rules, since no boilerplate code is needed to encode implicit relations or provide access to internal data structures. The `deploymentUnit` predicate creates a new ontology concept that matches a function instance executing on any virtualization technology. The use of abstract predicates such as `host` and `deploymentUnit` additionally enables operators to write reusable logic to serve multiple use cases. If an operator were to write a rule requiring deployments to be dispersed, this could be written to apply to any kind of deployment unit or execution environment.

Intent processing in SWIFT adopts the life-cycle model depicted in Figure 2. The life-cycle consists of six stages:

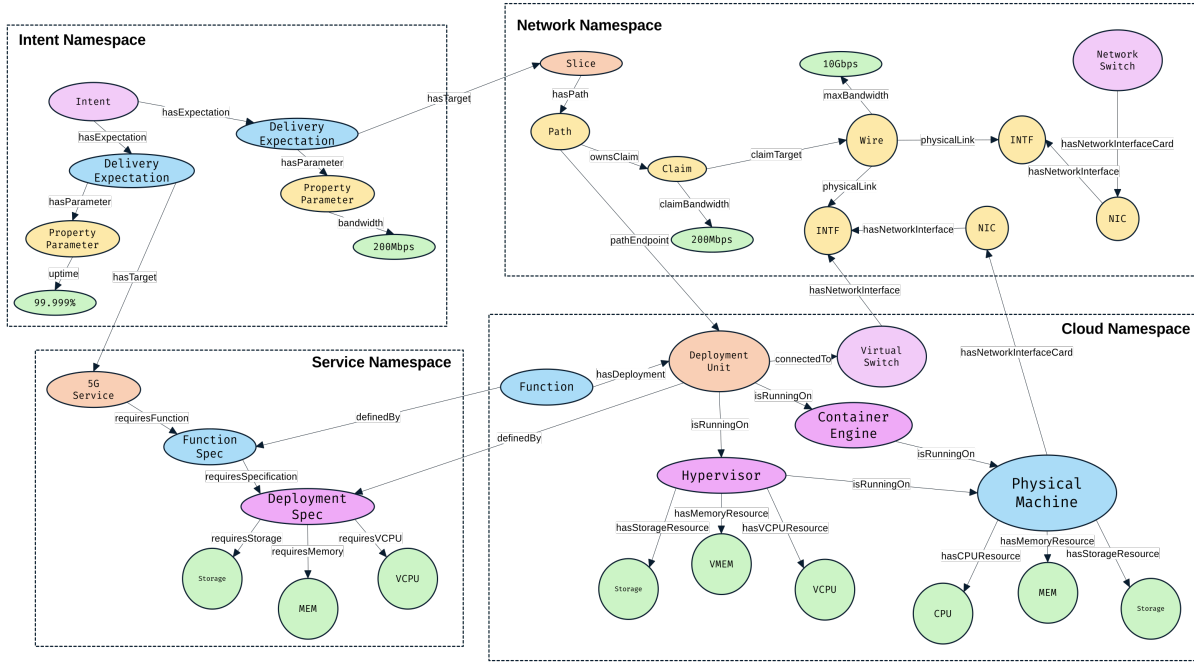


Fig. 3. The SWIFT network ontology consists of 4 namespaces: Intent, Network, Service and Cloud.

```

assert_redundant(_) :-
\+ (
    host(Host),
    hasDeploymentUnit(Func, DepU),
    hasFunction(FuncChain, Func),
    isRunningOn(ODepU, Hypr),
    hasDeploymentUnit(OFunc, ODepU),
    hasFunction(OFuncChain, OFunc),
    requiresFunctionChain(
        Service,
        FuncChain
    ),
    requiresFunctionChain(
        Service,
        OFuncChain
    ),
    serviceSpecification(Service),
    FuncChain \= OFuncChain
).

```

Listing 2: Example SWIFT intent validation predicate that checks the replication degree of a cloud instances.

creation, activation, planning, validation, deactivation, and destruction. The stages offer a model for operators to integrate their intent processing logic. The create and activate stages initialize the intent state and insert the appropriate associations in the knowledge graph. Similarly, the deactivation and destruction stages allow developers to gracefully remove the configuration associated with the intent from the knowledge graph and the infrastructure. Of particular interest are the planning and validation states, which allow an operator to implement rules that infer the state required to fulfill an intent goal and validate intent goal satisfiability in a dynamic network infrastructure.

To implement an intent scenario, a developer must provide

Prolog predicates that plan and validate intents. An intent planning predicate is depicted in Listing 3, which presents an example planning predicate for cloud resources in our intent handler implementation. Our slice delivery intent requires the deployment of a dedicated SMF and UPF instance. The predicate collects these requirements, and by performing a breadth-first search, it looks for the Hypervisor closest to the cells that service the slice Radio requirements and supports the required compute resources (MinCPU, MinMEM, MinSTOR).

Validation predicates are executed during the validation stage and encode logic that confirms the different intent guarantees. Effectively, they encode the intent SLA offered to users. Listing 2 presents an example interpretation of a “service uptime” intent goal, which specifies in 13 lines that replicant function chains within the same service should not reside on the same hypervisor, thereby limiting the effect of a rack failure on service uptime. Validation predicates are executed by the framework at fixed intervals, as well as when a new configuration is inferred by the ontology. If a predicate validation is false, then the associated intent object moves to the planning stage to compute an alternative deployment plan for the intent.

C. SWIFT domain adapters

Domain adapters allow operators to synchronize the knowledge graph with real-time data from the infrastructure and apply configuration updates. They are implemented using a mix of Prolog and Rust code, and they implement transformation logic, which enriches data models with metadata from our semantic ontology. The Prolog predicates are used to query and update the knowledge graph with the latest infrastructure state. An example of a Prolog query predicate can be found

```

solve_optimal :-
  nb_setval('OptimalQuality', 1000000),
  nb_setval('OptimalHosts', []),
  retractall(constraintHost(_)),
  retractall(constraintHop(_)),
  once(findall(V, (
    unresolvedHostVar(V),
    HostVars)
  )),
  intent:resource_discovery(CPU, MEM, STOR),
  network:assert_hosts,
  network:assert_hops,
  network:prune,
  findall(_, (
    discover_hosts(
      HostVars, [], CPU, MEM, STOR
    )), Results),
  network:retract_hostvar_claims(HostVars),
  (
    Results == [] -> (
      relax_constraints,
      findall(_, (
        optimal_solve_hosts(
          HostVars, [], MinCPU, MinMEM, MinSTOR
        )), _);
    true
  ),
  nb_getval('OptimalHosts', OptimalHosts),
  OptimalHosts \= [],
  deploy_devices(OptimalHosts).

```

Listing 3: Intent Planning Predicate. The predicate searches for a Hypervisor with sufficient resources to support the deployment of dedicated UPF and SMF functions for a slice.

```

dump_nsd(O) :-
  rdf(Name, icm:hasTarget, _),
  ID = Name,
  /* Description */
  rdf(I, rdf:type, im:'Intent'),
  rdf(I, rdfs:comment, AD),

  /* Get all inferred links */
  virtuallinkdesc(_, VLD),

  /* Get all vnfd */
  vnfdid(_, VNFDIDs),
  defaultf(VNFDIDs, DF),
  Version = '1.0',
  O = nsd{
    id: ID, description: AD,
    name: Name, version: Version,
    virtual_link_desc: VLD,
    vnfd_id: VNFDIDs, df: DF}.

```

Listing 4: Example SWIFT query that exports an OSM NSD for the cloud adapter.

in Listing 4. The predicate generates NFV-MANO Network Service Descriptors (NSD) for a network service using data from the Cloud namespace of the ontology. The query collects all the Virtual Network Function Descriptors (VNFD) and Virtual Link Descriptors (VLD) references inferred by the rule engine about an intent and combines them into a single NSD YAML object that can be deployed through the Open Source MANO (OSM) orchestrator. Domain adapters utilize Rust code to interact with external management APIs (e.g., OSM REST API). SWIFT framework offers built-in adapters for the ONOS intent interface, the OSM orchestrator, and the

Free5GC configuration, while network operators can extend them to support different technology or vendor interfaces.

D. SWIFT Implementation

Our proof-of-concept implementation of the SWIFT intent handler framework was developed using a combination of SWI-Prolog and Rust. Rust was chosen as an interface language primarily for its performance, library availability, and superior concurrency model to reduce latency in interacting with multiple APIs and orchestrators in parallel. In addition, our rust implementation offers an opportunity to build libraries for our Prolog engine, which, in future work, may be used to substitute Prolog's in-built RDF database for third-party triplestores, which could benefit future production use cases with powerful database features.

At the time of writing, the SWIFT code offers a cloud adapter that interfaces with the OSM orchestrator and the Kubernetes and OpenStack infrastructure managers. Additionally, a network adapter can collect topology information from an ONOS SDN controller and use its intent API to establish end-to-end paths on an SDN transport network. Finally, a 5G Core adapter allows intents to control the configuration of 5G core functions using OSM actions.

SWIFT offers out-of-the-box support for a slice management intent. The intent automates the deployment of a mobile slice on a set of radio stations. It deploys and configures a set of Session Management Functions (SMF) and UPF instances in an optimal cloud location using the OSM orchestrator, it updates the records of the Unified Data Management (UDM) to allow access to end-users and provisions the required paths between the radio sites and the selected cloud location. In future handler releases, we aim to also include support for radio configuration, using the Northbound Interface of the O-RAN Service Management and Orchestration (SMO) service.

V. EVALUATION

This section reports on the ability of the SWIFT platform to support common multi-domain automation scenarios, including the scalability of the semantic reasoner when optimizing the deployment plan of large numbers of intents. We use a dual-socket DELL Poweredge R750 (2xIntel 5320, 128GB RAM, Ubuntu 22.04) to execute our experiments. For our rule-based engine, we use SWI-Prolog 9.2.6.

In our experiments, we use the network topology scenario depicted in Figure 4, inspired by the design of BT's UK 21CN network [28]. This topology consists of three layers: access, metro, and core. The core network consists of 5 switches that connect different metro switches. Core switches offer connectivity to a large cloud data center. The metro area consists of 2 switches, which aggregate traffic from access switches to the core. Finally, the access layer uses fully connected pairs of switches to connect radio locations (gNodeBs) with the network. Using this blueprint, we create two topology variations: a small topology (1 core, 2 metro, 4 access, and 4 gNodeB) and a large topology (5 core, 40 metro, 80 metro, and 80 gNodeB).

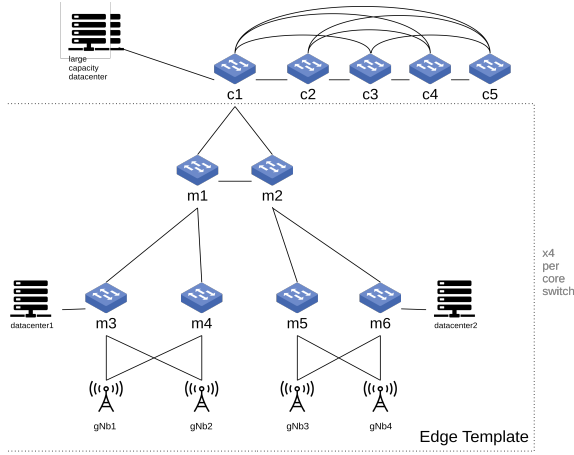


Fig. 4. Emulated topology of a mobile network. The topology is inspired by the BT 21CN UK network topology.

A. Automation support

This experiment evaluates the ability of the SWIFT platform to support complex automation scenarios in a 5G network. Specifically, we developed an intent scenario that deploys a mobile slice in a 5G network by configuring the transport, core, and cloud domains. For this experiment, we emulated a smaller variant of the topology in Figure 4. Our experiment used a set of OpenVSwitch instances, managed by an ONOS controller, to emulate the core, access, and metro switches. We also initialized and connected a Kubernetes Pod with the core topology switch, orchestrated by an OSM instance. The Kubernetes cluster operated a Free5GC [29] network service to emulate the 5G Core of a mobile network, and an iperf server, used to measure the network performance of the user devices. Additionally, we used UERANSIM [30] containers connected to the access switches, to emulate the radio and user devices. We execute an instance of our SWIFT network and connect it to the Kubernetes, OSM and ONOS services.

At startup, the mobile network uses a default slice (slice 1) to forward the iperf traffic from two UEs (UE1, UE2) connected to the same gNodeB to the iperf server on the Pod connected to the core switch. After 40 seconds of operation, we submit a new slice intent to the SWIFT instance. The SWIFT intent handler deploys dedicated UPF and SMF instances for a new slice on our second datacenter, connected to the metro switch m6. Additionally, the intent handler generates a new UDM and Access and Mobility Management Function (AMF) configuration that enables access for a UE to the new slice. Once the new UPF and SMF instances are up, the intent handler submits a set of HostToHost intents to the ONOS service to connect the new UPF and SMF instances to the 5G Core, the iperf server, and any gNodeB which is configured to access this slice. The slice requires approximately 15 seconds to activate. Once the slice is activated, a third UE is registered with a valid gNodeB, and runs a separate iperf session to the iperf server.

Figure 5 reports the throughput achieved by each iperf

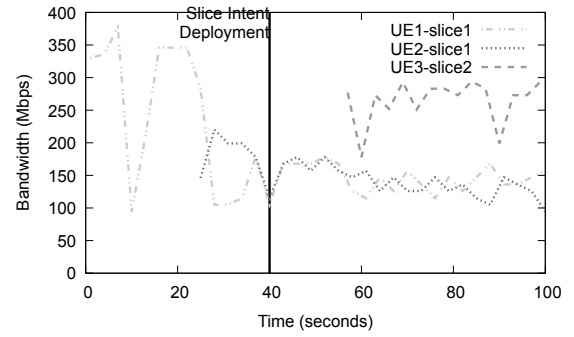


Fig. 5. Performance of two slices configured by the SWIFT intent handler. The bandwidth of the first slice is shared between UE1 and UE2. The second slice is created dynamically and used exclusively by UE3.

session for the duration of the experiment. The first UE reaches the configured limit of slice one very quickly. UE2 starts its session 25 seconds after the first session, leading the two instances to share the total bandwidth of slice one. Nonetheless, when UE3 is instantiated and starts the iperf client, the bandwidth limit of slice two is achieved with only a marginal impact on slice 1, demonstrating how the multi-domain configuration can optimize slice performance and ensure that the addition of a new slice does not influence throughput of existing slices. This experiment demonstrates the ability of SWIFT to realize network design decisions made by the intent handler's Decision component and implement inferred configurations using orchestration tools.

B. Platform Scalability

Network slices are the main mechanism for allocating resources in a mobile network. Controlling slice resources requires accurate allocations across domains to ensure end-to-end Quality of Service (QoS) goals, such as bandwidth guarantees. This includes allocating network paths for slice traffic in the transport domain with sufficient capacity to support the end-to-end bandwidth guarantee. Furthermore, it is good practice to create for each slice a dedicated UPF and SMF instance [31] with sufficient compute resources to meet the bandwidth guarantees of the slice. Our intent scenario implements slice optimization logic that models the resource allocation as a constraint satisfaction problem during the planning stage of the intent. Once a user submits a network slice intent, additional constraints are inferred based on operator-defined policies.

This experiment evaluates the scalability of the SWIFT semantic reasoner to generate deployment plans for a number of slice intent with a fixed bandwidth goal. For this experiment, we use the large topology in Figure 4, where each core is connected to 4 edge templates, totaling 24 metro switches, 8 datacenters, and 16 gNodeBs per core switch. To focus our evaluation on the performance of the SWIFT reasoner and the implementation of our intent processing logic, we employ a simple infrastructure simulation mechanism. We create a dummy domain adapter that imports the large topology infor-

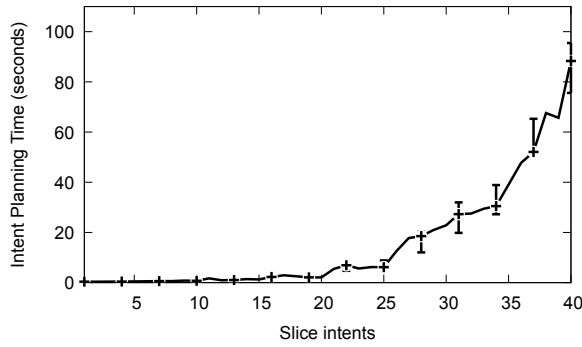


Fig. 6. Processing time required by the SWIFT semantic reasoner and intent logic to plan the deployment of a varying number of slice intents.

mation in the knowledge graph. We log the time to complete the planning of a configuration of a new slice intent, without applying any configuration to the underlying infrastructure.

We created a set of slice intents that model a 100Mbps slice request. Each intent is applied to all gNodeBs as a deployment location, resulting in 80 new slices created on each run. It is assumed that the slices are used to support edge services, requiring our intent logic to deploy dedicated UPF and SMF instances to an edge cloud location close to the gNodeB. At each gNodeB, a number of deployment options are identified and evaluated according to a fitness function that considers critical factors, such as the number of hops in the path, available resource capacity, and unallocated link capacity along identified routes. Once a deployment location is selected, the network slice generates a `Path` object to support the links connecting the gNodeB to the UPF and SMF pair. This `Path` object includes details on bandwidth allocations on each link. The allocation of links significantly influences deployment decisions, as our decision-making process prioritizes routes with lower allocations, rejecting those that are maximally allocated, in an effort to spread load across network links.

Figure 6 presents the processing time required by the semantic reasoner and our intent logic to plan the deployment of every slice request. We repeat the experiment 5 times and report in the figure the min, median and max intent planning time. The x-axis describes the number of intents submitted, which deploy a UPF and SMF pair for each gNodeB, resulting in 160 new deployments after each pass. The time required to identify and verify suitable deployment locations increases exponentially as users generate more slices and edge resources are reduced. This is due to several factors. Firstly, as slices are deployed, the nearest machines gradually approach max capacity, requiring the system to traverse and verify compute locations closer to the core of the network topology. Additionally, to avoid simply maximizing local resource utilization, our deployment function gradually shifts focus to core datacenter deployments as local machines become increasingly encumbered, leading to a greater number of link verifications and bandwidth calculations. It's important to note that this is a heuristic that could benefit greatly from further

optimizations. Our PoC system uses backward chaining logic exclusively to infer relationships between concepts at runtime. Modern graph database solutions enable forward chaining logic, which can infer concepts before running queries, which could enable our Prolog system to take shorter paths to run the same queries, shortening execution time. Furthermore, our Prolog rule engine uses a single core to perform calculations. Despite these challenges, our results indicate that our approach can adequately reason and deploy numerous network slice intents in a timely fashion. Our example intent scenario can plan the delivery of edge slice intents in under 100 seconds in a heavily utilized network infrastructure (40 slice intent per gNodeB deployed). This time is comparable to the deployment timescale of current orchestration frameworks, like OSM.

VI. CONCLUSION

The IBN paradigm continues to gain traction as an effective autonomic management approach, however, existing implementations remain narrowly tailored to specific use cases, limiting their extensibility. We observe that these limitations in general applicability arise from a lack of data modeling capabilities that support seamless model integration and contextual information processing for multi-domain IBN management. The paper presents the design implementation of SWIFT, an intent handler framework that prioritizes the use of extensible, semantic metadata in the modeling of knowledge domains. The framework adopts Semantic Web technologies to converge the information of different management data models using a network ontology. Furthermore, operators can use logic predicates to encode intent processing logic, which is enforced by a semantic reasoner during runtime. SWIFT's Prolog reasoner employs logic rules to effectively integrate the multiple knowledge domains of network state, creating a network knowledge graph, with which abstract intent goals can be contextualized. Our work illustrates the role knowledge graphs can play in enhancing a system's capability to interpret and respond to complex, abstract requirements.

In future work, we aim to explore two directions. Firstly, we want to explore how the design of the SWIFT architecture can encompass telemetry services and support run-time configuration adaption in response to monitoring event. Introducing support for telemetry information enables support for intent adaptation during runtime and in response to changing workload patterns, but it also increases the data dynamism of the intent handler context. Secondly, we want to explore the integration of AI/ML models and Digital Twin technologies to support the optimization and validation of intent handler decisions. AI techniques, including reinforcement learning and bayesian optimization, can be used to improve the scalability of intent planning.

ACKNOWLEDGMENT

This work is supported by the UK DSIT under the Future Open Networks Research Challenge project TUDOR. The views expressed are those of the authors and do not necessarily represent the project.

REFERENCES

- [1] 5G Global Launches & Statistics. Networks. [Online]. Available: <https://www.gsma.com/solutions-and-impact/technologies/networks/5g-network-technologies-and-solutions/5g-innovation/>
- [2] K.-L. Lee, C.-N. Lee, and M.-F. Lee, "Realizing 5G Network Slicing Provisioning with Open Source Software," in *2021 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, pp. 1923–1930. [Online]. Available: <https://ieeexplore.ieee.org/document/9689483>
- [3] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, "Intent-Based Networking - Concepts and Definitions," <https://www.rfc-editor.org/info/rfc9315>, Oct. 2022.
- [4] B. E. Carpenter, S. Jiang, and M. Behringer, "Autonomic networking integrated model and approach (anima) architecture," Internet Engineering Task Force, Internet-Draft draft-ietf-anima-architecture-16, March 2020, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/html/draft-ietf-anima-architecture-16>
- [5] IG1253A Intent Common Model v1.1.0. TM Forum. [Online]. Available: <https://www.tmforum.org/resources/how-to-guide/ig1253a-intent-common-model-v1-1-0/>
- [6] A. C. Baktir, A. D. N. Junior, A. Zahemszky, A. Likhyan, D. A. Temesgene, D. Roeland, E. D. Biyar, R. F. Ustok, M. Orlić, and M. D'Angelo, "Intent-based cognitive closed-loop management with built-in conflict handling," in *2022 IEEE 8th International Conference on Network Softwarization (NetSoft)*, pp. 73–78. [Online]. Available: <https://ieeexplore.ieee.org/document/9844074>
- [7] M. Xie, P. H. Gomes, J. Niemöller, and J. P. Waldemar, "Intent-Driven Management for Multi-Vertical End-to-End Network Slicing Services," in *2022 IEEE Globecom Workshops (GC Wkshps)*, pp. 1285–1291. [Online]. Available: <https://ieeexplore.ieee.org/document/10008673>
- [8] K. Mehmood, K. Kravetska, and D. Palma. Knowledge-based Intent Modeling for Next Generation Cellular Networks. [Online]. Available: <http://arxiv.org/abs/2302.08544>
- [9] A. Wong, P. Ray, N. Parameswaran, and J. Strassner, "Ontology mapping for the interoperability problem in network management," vol. 23, no. 10, pp. 2058–2068, 2005. [Online]. Available: <https://ieeexplore.ieee.org/document/1514535>
- [10] J. I. Ham, F. van der Dijkstra, F. Travostino, H. M. A. Andree, and C. de Laat, "Using RDF to describe networks," vol. 22, no. 8, pp. 862–867. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X06000410>
- [11] G. Pantuza, F. Sampaio, L. F. M. Vieira, D. Guedes, and M. A. M. Vieira, "Network management through graphs in Software Defined Networks," in *10th International Conference on Network and Service Management (CNSM) and Workshop*. IEEE, pp. 400–405. [Online]. Available: <http://ieeexplore.ieee.org/document/7014202/>
- [12] C. Villalonga, M. Strohschach, N. Snoeck, M. Sutterer, M. Belaunde, E. Kovacs, A. V. Zhdanova, L. W. Goix, and O. Droegehorn, "Mobile Ontology: Towards a Standardized Semantic Model for the Mobile Domain," in *Service-Oriented Computing - ICSSOC 2007 Workshops*, ser. Lecture Notes in Computer Science, E. Di Nitto and M. Ripeanu, Eds. Springer, pp. 248–257. [Online]. Available: <https://link.springer.com/chapter/10.1007/978-3-540-93851-4>
- [13] M. Bjorklund, "YANG - A Data Modeling Language for the Network Configuration Protocol (NETCONF)," Internet Requests for Comments, August 2016. [Online]. Available: <https://www.rfc-editor.org/info/rfc7950>
- [14] P. Berde, M. Gerola, J. Hart, Y. Higuchi, M. Kobayashi, T. Koide, B. Lantz, B. O'Connor, P. Radoslavov, W. Snow, and G. Parulkar, "ONOS: Towards an Open, Distributed SDN OS," in *Proceedings of the Third Workshop on Hot Topics in Software Defined Networking*, ser. HotSDN '14. New York, NY, USA: ACM, 2014, pp. 1–6. [Online]. Available: <https://dl.acm.org/doi/10.1145/2620728.2620744>
- [15] J. Medved, R. Varga, A. Tkacik, and K. Gray, "OpenDaylight: Towards a Model-Driven SDN Controller Architecture," in *Proceedings of the 2014 IEEE 15th International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*. Sydney, Australia: IEEE, 2014, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/6918985>
- [16] B. T. Loo, J. M. Hellerstein, I. Stoica, and R. Ramakrishnan, "Declarative routing: Extensible routing with declarative queries," in *Proceedings of the 2005 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*, ser. SIGCOMM '05. Association for Computing Machinery, pp. 289–300. [Online]. Available: <https://dl.acm.org/doi/10.1145/1080091.1080126>
- [17] B. T. Loo, T. Condie, M. Garofalakis, D. E. Gay, J. M. Hellerstein, P. Maniatis, R. Ramakrishnan, T. Roscoe, and I. Stoica, "Declarative networking: Language, execution and optimization," in *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data - SIGMOD '06*. ACM Press, p. 97. [Online]. Available: <http://portal.acm.org/citation.cfm?doid=1142473.1142485>
- [18] Z. B. Houidi, "A Knowledge-Based Systems Approach to Reason About Networking," in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*, ser. HotNets '16. Association for Computing Machinery, pp. 22–28. [Online]. Available: <https://doi.org/10.1145/3005745.3005758>
- [19] C. Rotsos, A. Farshad, D. King, D. Hutchison, Q. Zhou, A. J. G. Gray, C.-X. Wang, and S. McLaughlin, "ReasonNet: Inferring Network Policies Using Ontologies," in *2018 4th IEEE Conference on Network Softwarization and Workshops (NetSoft)*. IEEE, pp. 159–167. [Online]. Available: <https://ieeexplore.ieee.org/document/8460050/>
- [20] J. McNamara, E. Aumayr, L. Fallon, and E. Fallon, "A Flexible Interpreter For Intent Realisation," in *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, pp. 1–6.
- [21] J. Massa, S. Forti, F. Paganelli, P. Dazzi, and A. Brogi, "Declarative Provisioning of Virtual Network Function Chains in Intent-based Networks," in *2023 IEEE 9th International Conference on Network Softwarization (NetSoft)*, pp. 522–527. [Online]. Available: <https://ieeexplore.ieee.org/document/10175449>
- [22] D. Borsatti, W. Cerroni, and S. Clayman, "From Category Theory to Functional Programming: A Formal Representation of Intent," in *2022 IEEE 8th International Conference on Network Softwarization (NetSoft)*, pp. 31–36. [Online]. Available: <https://ieeexplore.ieee.org/document/9844061>
- [23] A. S. Jacobs, R. J. Pfitscher, R. H. Ribeiro, R. A. Ferreira, L. Z. Granville, W. Willinger, and S. G. Rao, "Hey, lumi! using natural language for Intent-Based network management," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, Jul. 2021, pp. 625–639. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/jacobs>
- [24] T. Forum, "TR290 Intent Common Model v3.0.0," TM Forum, Tech. Rep., April 2023. [Online]. Available: <https://www.tmforum.org/resources/reference/tr290-intent-common-model-v3-0-0/>
- [25] 3rd Generation Partnership Project (3GPP), "Management and orchestration; intent driven management services for mobile networks," 3GPP, Tech. Rep. TS 28.312, 2024. [Online]. Available: <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3554>
- [26] M. Dean, G. Schreiber, S. Bechhofer, F. van Harmelen, J. Hendler, I. Horrocks, D. L. McGuinness, P. F. Patel-Schneider, and L. A. Stein, "OWL Web Ontology Language Reference," World Wide Web Consortium (W3C) Recommendation, February 2004. [Online]. Available: <https://www.w3.org/TR/owl-ref/>
- [27] RDF - Semantic Web Standards. [Online]. Available: <https://www.w3.org/RDF/>
- [28] The Internet Topology Zoo. [Online]. Available: <http://www.topology-zoo.org/dataset.html>
- [29] free5GC, "free5gc: Open source 5g core network based on 3gpp r15," <https://github.com/free5gc/free5gc>, 2024, accessed: 2024-10-06.
- [30] A. Gungor, "Ueransim: Open source 5g ue and ran (gnoneb) simulator," <https://github.com/aligungr/UERANSIM>, 2024, accessed: 2024-10-06.
- [31] C.-C. Tsai, F. J. Lin, and H. Tanaka, "Evaluation of 5G core slicing on user plane function," *Communications and Network*, vol. 13, no. 03, pp. 79–92, 2021. [Online]. Available: <https://www.scirp.org/journal/paperinformation?paperid=111073>